Title Page

Title: Thread scheduling with GTthreads
Author: Barnabe Marty
Date: September 15, 2025

Abstract

The GTthreads library implements a user-level threading system that provides efficient scheduling across multiple kernel space threads (kthreads) mapped to CPU cores. The base library implements a priority-based scheduler where user space threads are organized in terms of a highest-priority-first policy. The workload for these user space threads (uthreads), in the original implementation, is simulated via matrix multiplication in which computational tasks are distributed across multiple uthreads, each responsible for computing portions of the result matrix.

While priority scheduling is somewhat efficient, it can lead to unfair resource allocation and starvation of lower priority uthreads. To address this, three additional enhancements were added:

Credit Scheduler: This scheduling algorithm assigns time credits to threads, and each thread consumes credits proportional to its execution time. This ensures fairness of scheduling, mitigating the starvation problems of the priority-based scheduler.

Cooperative Yielding (gt_yield): A voluntary yield mechanism in which threads relinquish CPU control at strategic points in their execution.

Load Balancing: The system incorporates load balancing capabilities to redistribute threads more evenly across available CPU cores, preventing scenarios of overloading cores while others remain idle.

Table of Contents

System Design

This section describes the overall design of the schedulers, cooperative yield, and load balancing. An additional figure for the priority scheduler is added in Appendix A as reference.

Priority Scheduler

The priority scheduler is the foundation of the GTthreads library, implementing a traditional highest-priority-first scheduling algorithm. The system organizes uthreads in a multi-dimensional runqueue structure where threads are categorized by both priority level and group membership.

The scheduler maintains two runqueues per kthread: an active queue and an expired queue. The active queue contains threads ready for execution, while the expired queue holds threads that have exhausted their scheduling opportunity in current round. When the active queue is empty, the scheduler swaps the two queues promoting expired queue to active status.

Thread selection follows a priority order bitmask for efficient priority level identification. The operation quickly identifies highest priority level in an O(1) complexity using a `LOWEST_BIT_SET` operation for the `TAILQ` data structure.

Credit Scheduler

The credit scheduler enhances fairness by introducing temporal resource allocation limits. Each thread is assigned a default credit allocation (25, 50, 75, 100), allowing for differentiated service levels across thread categories. When threads are created or recharged, they receive credits corresponding to their group's allocation. During the selection process, the scheduler looks for **the highest credited thread**[1] and runs it on the CPU. After execution of a thread, the actual runtime is tracked using `gettimeofday()` timestamps, and the credits proportional to execution time (**1 credit per millisecond** of execution for this design) are deducted.

Threads that exhaust their credits are migrated from the active queue to the expired queue, preventing them from monopolizing the CPU. Once all threads have exhausted their credits, the credit recharging system activates during the queue swap.

The credit scheduler was implemented by extending the existing GTthreads package while maintaining compatibility with the original priority scheduler. The base implementation involved modifications across multiple components to integrate credit-based fairness in the existing framework.

[1] Unlike the original Xen credit scheduler, which picks the first non-zero credited process, this version selects the highest-credited one, ensuring proportional fairness while preventing starvation as credits are consumed.

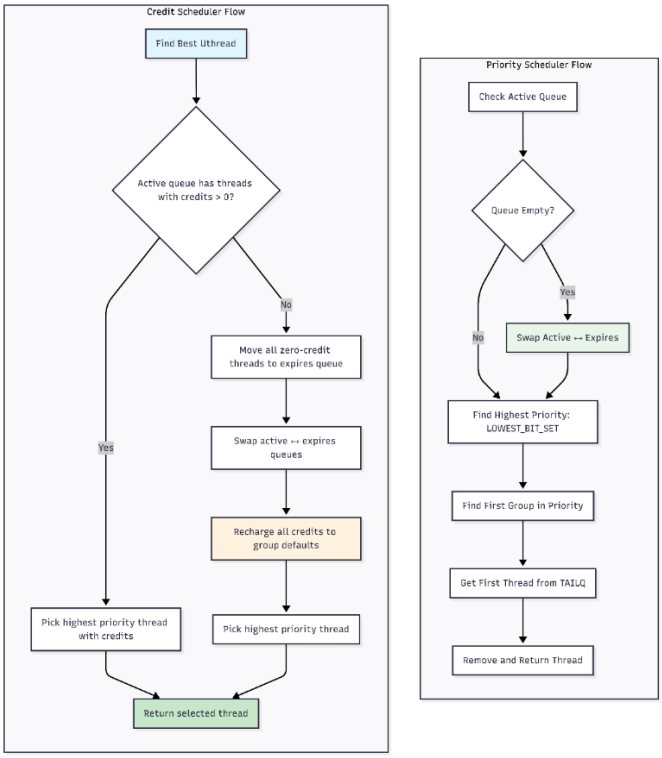


Figure 1: Flowchart comparing credit and priority-based scheduling decisions

Figure 1 depicts the very simple decision-making process that the credit-based scheduler makes compared to the priority-based scheduler. It is important to note that the credit-based scheduler approach does have O(n) complexity due to it iterating through the runqueue to find highest credited uthread.

Cooperative Yield

The `gt_yield()` function implements cooperative multitasking by allowing threads to voluntarily give up CPU control at strategic execution points. This mechanism is valuable for compute-intensive tasks, or long I/O blocking operations.

The yielding process is split up into key steps: first, if we are running on a credit scheduler, the thread's runtime is calculated, and credits are deducted. The thread is then put back in the active runqueue, and `sigsetjmp()` is called to save current execution context, enabling seamless resumption when thread is rescheduled.

After context saving, the scheduler invokes `sched_find_best_uthread()` function to select the next thread for execution (according to whichever scheduling policy is in place).

The point at which threads yield is a very critical one, and for this design, threads yield every `YIELD_INTERVAL` row (about every 50 rows) when performing matrix multiplication benchmark.

Load Balancing

Load balancing in the GTthreads library is focused on redistribution threads across available/idle CPU cores to prevent resource concentration. The implementation helps with evening out resource utilization across all hardware available.

When enabled, the load balancer is called when a kthread cannot find any runnable uthread assigned to them. In this case, the load balancer finds the kthread in the system with the highest amount of uthreads assigned (at least more than one uthread assigned to the victim kthread, to avoid ping-ponging threads around) and migrates a uthread from the victim kthread to the target one (expired or active runqueue depending on whichever one has a uthread available). After that, the new thread's CPU affinity information is updated, as well as the victim's kthread metadata.

This implementation focuses on dynamic load balancing during runtime, more towards evening out of workload during execution not during initialization.

Experiment

To observe whether all modifications are beneficial to the system, an experiment was conducted on the system to quantify performance.

Setup

The experiment involved running 128 uthreads with 4 credit groups and 4 matrix sizes (where each uthread work on a matrix of its own), and cooperative yielding is disabled. Matrix sizes range in {32, 64, 128, 256} and credits group are {25, 50, 75, 100}. A total of 16 possible combinations of 4 credit groups and 4 matrix sizes were possible. Total execution time, wait time, and CPU time were all recorded in microseconds for analysis.

Results

For the credit scheduler described above, the graphs demonstrate the scheduler's behavior under the given experimental setup.

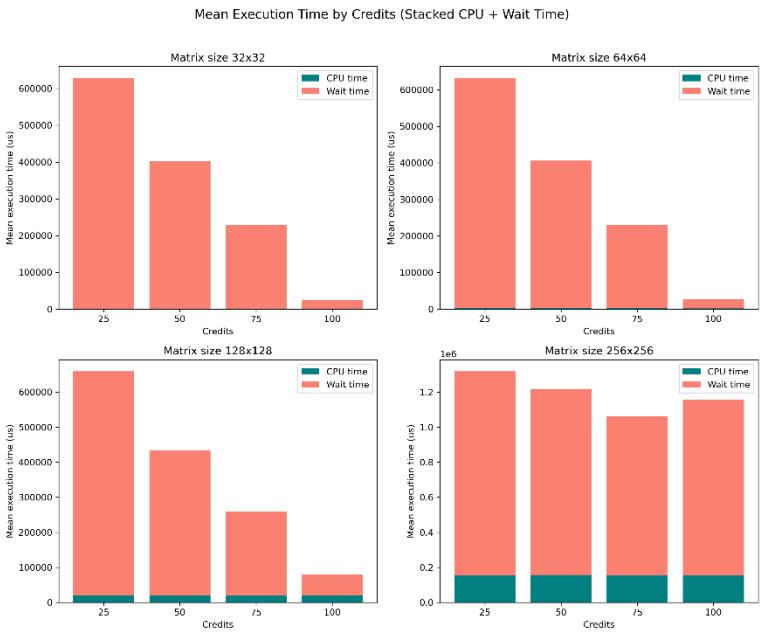


Figure 2: Credit Scheduler performance with given experimental setup (No Load Balancing)

To analyze the effects of load balancing, the same experiment was run on the same machine, but with the load balancing module enabled.

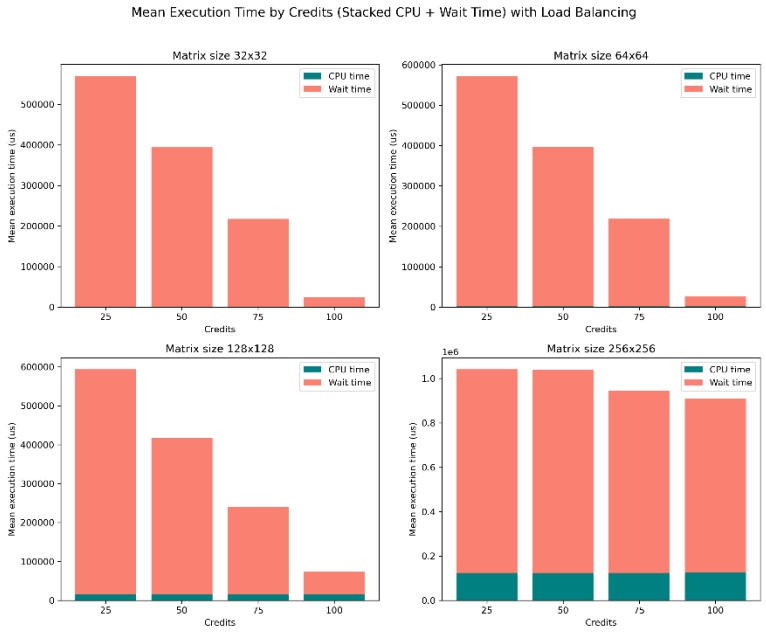


Figure 3: Credit Scheduler performance with given experimental setup and Load Balancing enabled

Analysis

Credit Scheduler Performance

The credit scheduler demonstrates robust performance characteristics that align with the expectations for fair-share scheduling systems. The data confirms an inverse relationship between credit allocation and total execution time across all workloads. This is especially effective when looking at small workloads (32x32 matrices in this case in Figure 2) where execution time decreases from about 640 milliseconds at 25 credits to 30 milliseconds at 100 credits, representing the fact that processes with a larger number of credits get more CPU time allocated to them, and therefore have a smaller total execution time.

The scheduler's credit accounting mechanism, implementing policy of one credit is equal to one millisecond of execution time[2], provides a granular approach that ensures resources are distributed proportionally to credit allocation while preventing monopolization by any single thread, as seen by the relatively inverse proportional trend described earlier.

The ratio of wait time to CPU time remains high across all configurations, typically exceeding 95% of total execution time. This is first a sign of the nature of the experiment (high workloads with lots of

[2] This is a design choice and a parameter we can change. For this design the time quantum was chosen to be 1ms as it seemed to have the best performance for this application.

threads contending for limited resources), but it also suggests potential opportunities for optimization in scenarios where maximum throughput matters more than fairness in scheduling

Load Balancing Performance

A comparative analysis between the results with and without load balancing shows a slight trend of more effective throughput across all tested scenarios (~12.5%[3] increase in throughput). The execution profile between results in Figure 2 (without load balancing) and Figure 3 (with load balancing) demonstrates that generally execution time slightly decreases with load balancing. This is to be expected as we are allowing for maximum resource utilization and not overloading one CPU core but rather have a more even distribution of workload across hardware.

The absence of an extremely substantial performance improvement (i.e. more than 20% increase in throughput) is due to the nature of the experiment again (limited resources, high contention between threads). There is an argument to be made that the matrix multiplication tasks appear to be mostly well-distributed across available processing cores without load balancing that it is negating the minimal advantages of explicit load balancing.

Comparison of Scheduler Performance

The credit scheduler implementation demonstrates significant advantages over the traditional O(1) priority scheduler in general. With the evaluation conducted, Figure 2 clearly shows a fair scheduler like how the priority scheduler would work but eliminating the worry of having a process starve the CPU. The predictability of the credit scheduler allows for good scalability of the system as can be seen from the results of the experiment (increasing matrix size generally makes scheduler keep up with fairness).

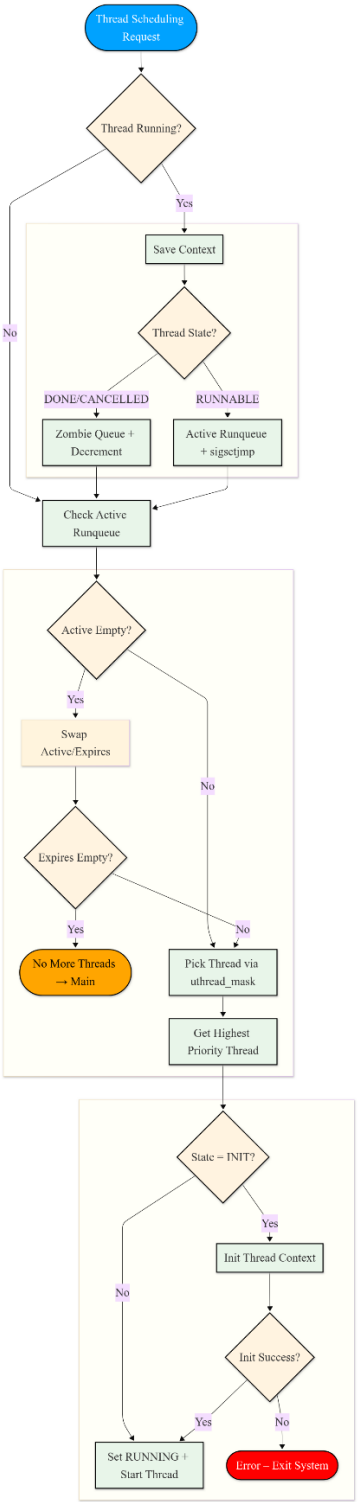
The only concern with the credit scheduler is when the system scales to much higher workloads. It appears that once the workload of the system reaches a threshold, the fairness of the scheduler breaks down (as shown with the highest matrix size workload in Figure 2). This is quite concerning as the overhead of having an O(N) credit scheduler may not be worth implementing if we have extremely high workloads. This, however, could be due to necessary optimizations in the current credit scheduler design, or perhaps in a different approach at how the credit scheduler picks the next thread to run (first non-zero credit thread instead of highest credited thread).

Overall, the results indicate that credit scheduling is more effective than priority scheduling when the primary objective for the system is fair resource allocation with support for priority. The generally inversely proportional trend between credits and execution time, and its predictability with high workload and low resource systems makes it extremely useful and robust.

[3] Looking at approximates for 64x64 matrix sizes between Figure 1 and Figure 2:
$\frac{560000-640000}{640000} \times 100 = 0.125 \times 100 = 12.5\%$

Appendix

Appendix A (GTthreads priority scheduler diagram)



Appendix B (printed credits before/after yield)

```
**************************************************
Run queue(ACTIVE_BEFORE_YIELD) state (CPU #0) :
**************************************************
uthreads details - (tot:3 , mask:10000)
**************************************************
uthread credits : T2(1c) T10(1c) T14(1c)
**************************************************

[gt_yield] credits tid=6 gid=0, creds=5, runtime_ms:3, runtime_us:3296

[gt_yield] deducted credits tid=6 gid=0, old_creds=5, new_creds=2

**************************************************
Run queue(ACTIVE_AFTER_YIELD) state (CPU #0) :
**************************************************
uthreads details - (tot:4 , mask:10000)
**************************************************
uthread credits : T2(1c) T10(1c) T14(1c) T6(2c)
**************************************************
```

Appendix C (printed queue of credits with load balancing)

```
[LOAD_BALANCE] ===== QUEUE STATUS BEFORE LOAD BALANCING =====
[LOAD_BALANCE] Victim CPU 0 - Active: 3 uthreads, Expires: 0 uthreads

**************************************************
Run queue(ACTIVE_VICTIM) state (CPU #1) :
**************************************************
uthreads details - (tot:3 , mask:10000)
**************************************************
uthread credits : T6(25c) T15(25c) T24(25c)
**************************************************

[LOAD_BALANCE] Successfully stole uthread 6 (group 0) from CPU 0 to CPU 1

[LOAD_BALANCE] ===== QUEUE STATUS AFTER LOAD BALANCING =====
[LOAD_BALANCE] Victim CPU 0 - Active: 2 uthreads, Expires: 0 uthreads

**************************************************
Run queue(ACTIVE_VICTIM) state (CPU #1) :
**************************************************
uthreads details - (tot:2 , mask:10000)
**************************************************
uthread credits : T15(25c) T24(25c)
**************************************************
```